SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.Gayathri	**Reg. No.:**	192472074
Section / Batch:	CSE–AI	**Date:**	21-10-2026

Code of Conduct

I M.Gayathri(192472074)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:M.Gayathri

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

Question 3 – Error Analysis of Stemming in News Article Classification

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant.

Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
   if word.endswith("s"):
      return word[:-2], "Plural Noun"
   else:
      return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
   print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
```

```
stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Morphological Analysis Error in a Biomedical Dataset

Aim

To analyze errors caused by Porter Stemming in biomedical words and understand the difference between inflectional and derivational morphology.

1. Morphological analysis

Word	Root	Affix	Type
infection	infect	-ion	Derivational
infectious	infect	-ious	Derivational
infected	infect	-ed	Inflectional
infect	infect	None	Root word

2. Apply Porter Stemming

The Porter Stemmer applies suffix-removal rules to each word.

Original word	Porter stem	Analysis
infection	infect	Related form reduced to root
infectious	infecti	Root not preserved correctly

infected	infect	Inflectional suffix removed
infect	infect	No change

3. Error identification and justification

- infection and infected both become infect, which may help retrieve related documents.
- infectious becomes infecti, which is not the intended root.
- Derivational suffixes can change meaning or word class, so blindly removing them can lose important information.
- Inflectional suffixes generally represent grammatical changes.

4. Python demonstration

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", ps.stem(word))
```

OUTPUT:

```
= RESTART: C:/Users/Madaraboina
infection -> infect
infectious -> infecti
infected -> infect
infect -> infect
```

Result

Porter Stemming reduces some related words to a common stem but may produce invalid roots. Lemmatization and biomedical dictionaries can improve clinical search accuracy.

Q4. Morphological Error Analysis in an Information Retrieval System

Aim

To analyze stemming errors in an e-commerce search engine and suggest an improved preprocessing strategy.

1. Identify the suffixes

Word	Decomposition	Type
watches	watch + es	Inflectional
watching	watch + ing	Inflectional
washable	wash + able	Derivational
washer	wash + er	Derivational
washed	wash + ed	Inflectional

2. Error analysis

- watches, watching, and washed contain grammatical suffixes in the intended interpretations.
- washable describes something that can be washed.
- washer refers to a device or an agent.
- Removing all suffixes without considering meaning may make product-search results less relevant.

3. Improved preprocessing strategy

1. Convert text to lowercase.
2. Tokenize product descriptions.
3. Use lemmatization for suitable inflectional forms.
4. Preserve important derivational forms.
5. Use product-specific synonyms and contextual information.

4. Python demonstration

```
from nltk.stem import PorterStemmer, WordNetLemmatizer

ps = PorterStemmer()
lemma = WordNetLemmatizer()

words = ["watches", "watching", "washable", "washer", "washed"]

for word in words:
    print(word, "-> Stem:", ps.stem(word),
          "| Lemma:", lemma.lemmatize(word, pos="v"))
```

Sample output:

```
========== RESTART: (
watches -> Stem: watch | Lemma: watch
watching -> Stem: watch | Lemma: watch
washable -> Stem: washabl | Lemma: washable
washer -> Stem: washer | Lemma: washer
washed -> Stem: wash | Lemma: wash
```

5. Justification

The stemmer may shorten a word without preserving its meaning. Lemmatization can return a dictionary form, while derivational words should be preserved when they represent different product concepts.

Result

Morphology-aware preprocessing helps maintain product meaning and improves the relevance of e-commerce search results.

Q6. Error Analysis of a Finite-State Morphological Parser

Aim

To identify logical errors in a plural noun parser and correct it to handle regular, -es, -ies, and irregular plurals.

1. def parser(word):

    if word.endswith("s"):

        return word[:-2], "Plural Noun"

    else:

        return word, "Singular"

Original code and errors

Errors identified:

1. word[:-2] removes two characters from every word ending in s.
2. The parser does not handle the -ies rule.
3. The parser does not recognize irregular plurals such as children.
4. It incorrectly assumes words ending in s are always plural.

2. Intermediate steps

Input	Rule	Calculation	Correct output
cars	Regular -s	cars[:-1]	car
boxes	-es	boxes[:-2]	box
cities	-ies	cities[:-3] + "y"	city
children	Irregular dictionary	children → child	child

For example, the original code processes cars as cars[:-2], giving ca. The corrected rule removes one character to obtain car.

3. Corrected Python program

```
def parser(word):
    word = word.lower()

    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "geese": "goose",
        "people": "person"
    }

    if word in irregular:
        return word, irregular[word], "Irregular Plural"

    if word.endswith("ies") and len(word) > 3:
        return word, word[:-3] + "y", "Plural Noun"

    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word, word[:-2], "Plural Noun"

    if word.endswith("s") and not word.endswith("ss"):
        return word, word[:-1], "Plural Noun"

    return word, word, "Singular or Unknown"


words = ["cars", "boxes", "cities", "children"]

for word in words:
    original, root, label = parser(word)
    print(original, "->", root, ":", label)
```

4. Sample output

```
==================================== RESTART
cars -> car : Plural Noun
boxes -> box : Plural Noun
cities -> city : Plural Noun
children -> child : Irregular Plural
```

5. Justification

The corrected parser checks irregular forms first, then applies the -ies, -es, and regular -s rules. This prevents incorrect character removal for the tested words.

Result

The corrected parser successfully handles the four test cases. However, a real morphological parser needs a larger lexicon and additional rules for exceptional forms.

Q7. Error Analysis of Morphological Feature Extraction

Aim

To correct a text preprocessing program that performs stemming after feature extraction and calculate the reduction in vocabulary size.

1. Original code error

```
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)



features = [

   stemmer.stem(word)

   for word in vectorizer.get_feature_names_out()

]
```

2. Intermediate vocabulary calculation

Input documents:

1. running runners runs

2. studies studied studying
3. organization organized organizer

Before stemming, each document has three unique words.

$$V_{before} = 3 + 3 + 3 = 9$$

After stemming:

Original words	Stemmed forms
running, runners, runs	run, runner, run
studies, studied, studying	studi, studi, studi
organization, organized, organizer	organ, organ, organ

The unique vocabulary becomes:

organ, run, runner, studi

Therefore:

$$V_{after} = 4$$

3. Vocabulary reduction calculation

$$Reduction = \frac{V_{before} - V_{after}}{V_{before}} \times 100$$
$$= 55.56\%$$

4. Corrected Python program

```
import re
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
```

```
    "organization organized organizer"
]

def stem_document(text):
    words = re.findall(r"[a-zA-Z]+", text.lower())
    return " ".join(stemmer.stem(word) for word in words)

# Before correction
vectorizer1 = CountVectorizer()
X1 = vectorizer1.fit_transform(documents)

# After correction: stem before feature extraction
processed_documents = [stem_document(d) for d in documents]

vectorizer2 = CountVectorizer()
X2 = vectorizer2.fit_transform(processed_documents)

print("Vocabulary before:",
      list(vectorizer1.get_feature_names_out()))
print("Size before:", len(vectorizer1.get_feature_names_out()))

print("\nProcessed documents:", processed_documents)
print("Vocabulary after:",
      list(vectorizer2.get_feature_names_out()))
print("Size after:", len(vectorizer2.get_feature_names_out()))
```

5. Sample output

```
========== RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DSA0304-NLP/C02-AT3/Q4.py ==========
Vocabulary before: ['organization', 'organized', 'organizer', 'runners', 'running', 'runs', 'studied', 'studies', 'studying']
Size before: 9

Processed documents: ['run runner run', 'studi studi studi', 'organ organ organ']
Vocabulary after: ['organ', 'run', 'runner', 'studi']
Size after: 4
```

6. Justification

Stemming before vectorization allows related word forms to share vocabulary features. In this example, vocabulary size decreases from 9 to 4, a reduction of 55.56%.

This reduces the number of features, but improved classification accuracy is not guaranteed. Accuracy must be measured using a labeled dataset and an actual classifier.

Result

The corrected preprocessing pipeline stems words before feature extraction and produces a smaller, consistent vocabulary for NLP tasks.